

April
27
2026.

Yahy Farah.

Assignment 7 Graphs

1 (text) Dijkstra, find the shortest path from Anya (human) to all other nodes using Dijkstra's (show your work and intermediate steps).

∴ So I start at Anya → Anya is 0.

Visited [].

anya → Frog = 0.1

anya → Orca = 1

anya → Dog = 4

anya → Penguin = 2

[Anya, Frog]

Frog has no outgoing path.

Anya will visit the orca.

[Any, Frog, Orca]

anya → orca → elephant = 4

anya → orca → wolf = 6

anya → orca → Rhino = 2

[Anya, Frog, Orca, Penguin]

Anya → Penguin → Dog = 4

Dog is already 4, so I don't update it.

[A, F, O, P, Rh, Elephant]

A → O → Rh → Elephant = 2.5

A → O → Rh → Bear = 14

* Elephant changes from 4 to 2.5 because it's shorter.

[A, F, O, P, Rh, Ele]

A → O → Rh → El → P = 5.5

Penguin is already 2, so no change.

[A, F, O, P, Rh, El, Dog]

A → D → Rabbit = 7

Visited →

[A, F, O, P, Rh, El, Dog, Ra]

A → D → Ra → F = 9

A → D → Ra → Otter = 12

* Frog is 0.1, so I do not update.

[A, F, O, P, Rh, El, D, Ra, Otter]

A → D → Ra → Otter → D = 13

A → D → Ra → Otter → Bear = 13.5

* Dog is already 4, so I keep it.

Bear becomes 13.5 because 13 < 14

[A, F, O, P, Rh, El, D, Ra, Otter, Bear]

A → D → Ra → Otter → Bear → Otter = 15.2

A → D → Ra → Otter → Bear → Sheep = 14

* Otter is already 12, so no change.

[A, F, O, P, Rh, El, D, Ra, Otter, Bear, Sheep]

A → D → Ra → Otter → Bear → Sheep → Otter = 18

A → D → Ra → Otter → Bear → Sheep → El = 15

A → D → Ra → Otter → B → SA → Wolf = 15

* I keep Wolf = 6 because it is cheaper.

Shortest path

Anya → F : 0.1

Anya → Orca = 1

Anya → Penguin = 2

Anya → Dog = 4

Anya → Orca → Rhino → Elephant : 2.5

Anya → Orca → Wolf = 6

anya → Dog → Rabbit = 7

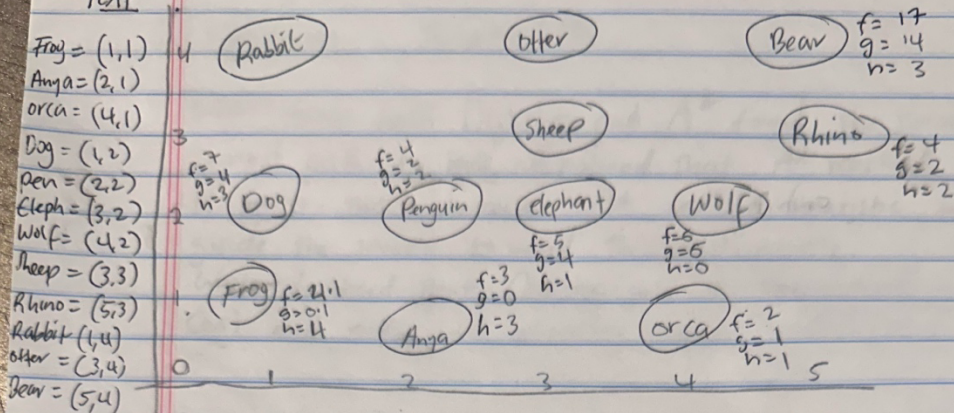
Anya → Dog → Rabbit → Otter = 12

Anya → Dog → Rabbit → Otter → Bear = 13.5

Anya → Dog → Rabbit → Otter → Bear → Sheep = 14.

Text

I give each node an X, y position



$$h = |x_{\text{current}} - x_{\text{dest}}| + |y_{\text{current}} - y_{\text{dest}}|$$

* Anya neighbors

Step 1 (dest = Wolf)	Frog (1,1)	Penguin (2,2)
Current (2,1)	g = 0.1 h = 1-4 + 1-2 = 4 f = 0.1 + 4 = 4.1	g = 2 h = 2-4 + 2-2 = 2 f = 2 + 2 = 4
g = 0 h = 2-4 + 1-2 = 3 f = 0 + 3 = 3	Orca (4,1) g = 1 h = 4-4 + 1-2 = 1 f = 1 + 1 = 2	Dog (1,2) g = 4 h = 1-4 + 2-2 = 3 f = 4 + 3 = 7

Smallest out of all neighbors f's

next I visit Orca.

Step 2

Orca has 3 outgoing edges so I want to compute each one	Wolf (4,2)	Rhino (5,3)
Elephant (3,2)	g = 1 + 5 = 6 h = 0 because it's my destination node f = 6	g = 1 + 1 = 2 h = 5-4 + 3-2 = 2 f = 2 + 2 = 4

* So I compare all current nodes and see who has

smaller f.

Rhino = 4
Penguin = 4

Tie rule
if f are ties
pick smaller h.

∴ Shortest path to wolf is

Any → Orca → Wolf = 6

Step 3 Rhino

* So I keep repeating the steps until I get to my destination node.

③ Comparison Anya $\rightarrow$ Wolf

Even Though both Dijkstra and A^* found the same shortest path to Wolf, I noticed that A^* reached it faster. This is because A^* uses a heuristic to guide the search toward the destination, while I noticed that Dijkstra just explores based on Cost and ends up checking more nodes.

Q4

For this algorithm, the time complexity is $O(n^3)$, where N is the total number of nodes in the graph. This is because the program uses three nested for loops that each iterate n times to check every possible connection between nodes. The reason is that for every node we pick as a "middleman," it must scan the entire N times N matrix to see if that node creates a new bridge between any other two points, leading to $N \times N \times N$ total operations.

The space complexity is $O(n^2)$, where n is the number of nodes in the graph. I arrived at this because the program initializes a 2D Boolean array to store the connectivity data. The reason is that representing a graph with n nodes in an adjacency matrix format requires a grid of n rows and n columns to track every possible link, meaning the memory used grows by the square of the number of nodes.

Q5

For this algorithm, the time complexity is $O(V * b^3)$, where V is the number of vertices and b is the branching factor. This is because the program still performs a depth-limited search starting from every vertex. The reason is that for each starting

character, i only traverse actual edges stored in the HashMap, exploring up to 4 steps away to see if we can close the loop back to the beginning.

The space complexity is $O(V + E)$, where V is the number of vertices and E is the number of edges. And this is because I am using an Adjacency List. The reason is also that the HashMap only stores the characters and their actual outgoing connections, which is much more efficient than a matrix, and the recursion stack only takes up a tiny amount of memory since it never goes deeper than 4 levels.

Q6

The time complexity is $O(n^2)$, where n is the number of vertices, because we use nested loops to print the adjacency matrix.

The space complexity is $O(n^2)$, where n is the number of vertices, because we store the graph using a matrix of size $n \times n$.